

0.1 Подбор гиперпараметров: определение

В машинном обучении различают два типа переменных:

- **Параметры модели:** это веса, которые алгоритм находит самостоятельно в процессе обучения на данных (например, коэффициенты w в линейной регрессии).
- **Гиперпараметры:** это внешние настройки алгоритма, которые задаются исследователем до начала обучения. Они определяют саму структуру модели и способ её обучения (например, глубина дерева, количество соседей в KNN, сила регуляризации).

Подбор гиперпараметров — это процесс поиска таких значений, при которых модель показывает наилучшее качество на новых данных.

0.2 Ключевые гиперпараметры для популярных алгоритмов

Ниже приведен перечень основных гиперпараметров, которые чаще всего настраиваются для достижения лучшего качества моделей.

Линейные модели (Linear/Logistic Regression): • **C** или **alpha:** Коэффициент регуляризации. Чем меньше значение (для C), тем сильнее регуляризация.

- **penalty:** Тип штрафа (l1, l2, elasticnet).
- **solver:** Алгоритм оптимизации (liblinear, saga, lbfgs).

Метод k-ближайших соседей (k-NN): • **n_neighbors:** Число соседей k .

- **weights:** Весовая функция (uniform или distance).
- **p:** Параметр метрики Минковского (1 — Манхэттен, 2 — Евклид).

Деревья решений (Decision Trees): • **max_depth:** Максимальная глубина дерева (контроль переобучения).

- **min_samples_split:** Минимум объектов, необходимых для разделения узла.
- **min_samples_leaf:** Минимум объектов в листе.
- **criterion:** Критерий ветвления (gini, entropy).

Случайный лес (Random Forest): • **n_estimators:** Количество деревьев в лесу.

- **max_features:** Число признаков для поиска лучшего разбиения.
- *Плюс все параметры одиночного дерева.*

Градиентный бустинг (XGBoost, LightGBM, CatBoost): • **learning_rate:** Скорость обучения (шаг градиентного спуска).

- **n_estimators:** Количество итераций (деревьев).
- **max_depth:** Глубина деревьев.
- **subsample:** Доля выборки для обучения одного дерева.
- **reg_alpha / reg_lambda:** L1 и L2 регуляризация.

Метод опорных векторов (SVM): • **C:** Параметр штрафа за ошибки (баланс между отступом и точностью).

- **kernel:** Тип ядра (linear, rbf, poly).

- **gamma**: Коэффициент для нелинейных ядер.

Нейронные сети: • **learning_rate**: Скорость обучения.

- **batch_size**: Размер пакета данных.
- **optimizer**: Алгоритм оптимизации (Adam, SGD, RMSprop).
- **dropout_rate**: Коэффициент исключения нейронов (регуляризация).

0.3 Методология подбора

Чтобы оценка качества была честной, нельзя подбирать гиперпараметры на тех же данных, на которых модель обучается или на которых она окончательно тестируется.

0.3.1 Схема Train-Validation-Test

Классический подход подразумевает разделение всего датасета на три независимые части. Это необходимо, чтобы избежать «переобучения под тест»: если мы будем подбирать параметры, ориентируясь на тестовую выборку, мы получим слишком оптимистичную оценку, которая не подтвердится на реальных данных (эффект "подгонки" под тест).

- **Train (Обучающая)**: 60–80% данных. Используется исключительно для настройки внутренних весов модели (параметров).
- **Validation (Валидационная)**: 10–20% данных. Служит для оценки качества при разных гиперпараметрах. Именно по результатам на этой части мы выбираем лучший набор настроек.
- **Test (Тестовая)**: 10–20% данных. «Золотой стандарт» оценки. К этой части обращаются ровно один раз в самом конце.

Важное правило: если после оценки на Test вы решили «еще немного подкрутить» параметры, Test автоматически становится частью Validation, и вам нужны новые данные для честного финального теста.

Типичный процесс подбора:

1. Фиксируем конкретный набор гиперпараметров.
2. Обучаем модель на **Train**.
3. Замеряем метрику качества на **Validation**.
4. Повторяем шаги 1–3 для всех интересующих комбинаций.
5. Выбираем комбинацию с лучшим результатом на валидации.
6. (Опционально) Переобучаем финальную модель на объединенной выборке **Train + Validation**.
7. Делаем итоговый замер качества на **Test**.



Рис. 1: Схема разделения данных для честной оценки.



Рис. 2: Сравнение Grid Search и Random Search.

0.4 Кросс-валидация (k-Fold)

Если данных недостаточно для выделения отдельной валидационной выборки или мы хотим получить более устойчивую оценку качества, используется **k-Fold кросс-валидация**.

Принцип работы:

1. Обучающая выборка делится на k равных частей (фолдов).
 2. Модель обучается k раз. В каждой итерации один фолд используется как валидационный, а остальные $k - 1$ — как обучающие.
 3. Итоговая метрика — это среднее арифметическое результатов на всех k валидационных фолдах.
- **Stratified k-Fold:** Вариант для классификации, где в каждом фолде сохраняется то же процентное соотношение классов, что и во всей выборке. Это критично при сильном дисбалансе классов.
 - **Плюсы:** Более надежная оценка качества (низкая дисперсия метрики), использование всех данных и для обучения, и для валидации.
 - **Минусы:** Вычислительная сложность (модель нужно обучить k раз).

0.5 Grid Search (Поиск по сетке)

Принцип: Исследователь задает дискретный список значений для каждого гиперпараметра. Алгоритм строит декартово произведение этих списков и перебирает **абсолютно все возможные комбинации**.

- **Плюсы:**
 - Исчерпывающий поиск в заданных границах.
 - Простота интерпретации и воспроизводимость.

- **Минусы:**

- «Проклятие размерности»: количество комбинаций растет экспоненциально с добавлением новых параметров.
- Неэффективность: много времени тратится на перебор «неважных» параметров, которые слабо влияют на результат.

0.6 Random Search (Случайный поиск)

Принцип: Вместо фиксации сетки мы задаем распределения (например, равномерное или логарифмическое) для параметров. Значения выбираются случайно.

Почему это работает (согласно Bergstra & Bengio): В большинстве задач лишь небольшое количество гиперпараметров являются действительно важными. Grid Search тратит время на перебор неважных измерений. Random Search же исследует гораздо больше уникальных значений «важных» параметров за то же количество итераций.

- **Плюсы:**

- Значительно быстрее при большом количестве параметров.
- Не привязан к конкретному шагу сетки.
- Вероятностная гарантия: при $n = 60$ итерациях мы с вероятностью 95% найдем решение, входящее в топ-5% лучших в пространстве поиска.

- **Минусы:**

- Не гарантирует нахождение глобального максимума (может «промахнуться» мимо узкого пика).
- Результат зависит от случайности (нужен фиксированный `random_state`).

0.7 Продвинутые методы подбора

Когда Grid и Random Search становятся слишком дорогими, на помощь приходят адаптивные методы, которые «учатся» на результатах предыдущих итераций.

0.7.1 Байесовская оптимизация

Основная идея — не искать вслепую, а построить вероятностную модель зависимости целевой метрики от гиперпараметров.

- **Суррогатная модель (Surrogate Model):** Аппроксимирует реальную (дорогую) целевую функцию. Чаще всего используются *Гауссовские процессы* или *Random Forest*. Модель предсказывает не только среднее значение метрики в точке x , но и дисперсию (неопределенность) $\sigma(x)$.
- **Функция полезности (Acquisition Function):** Решает, какую точку исследовать следующей. Она балансирует между:
 - *Exploitation* (Эксплуатация): Проверка областей, где модель предсказывает высокий результат.
 - *Exploration* (Исследование): Проверка областей с высокой неопределенностью (где мы еще не были).

- **Популярные функции:** **EI** (Expected Improvement) — математическое ожидание улучшения, **UCB** (Upper Confidence Bound) — верхняя граница доверительного интервала.

0.7.2 Алгоритм TPE (Tree-structured Parzen Estimator)

Это «облегченный» и эффективный вариант байесовской оптимизации, используемый в Optuna и Hyperopt.

Интуиция: Вместо моделирования функции $P(y|x)$ (как в Гауссовских процессах), TPE моделирует плотность распределения параметров $P(x|y)$.

1. Все результаты делятся на две группы: «лучшие» (верхний квантиль) и «остальные».
2. Строятся два распределения: $l(x)$ для лучших и $g(x)$ для остальных.
3. Следующей выбирается точка, которая максимизирует отношение $l(x)/g(x)$ — то есть точка, которая с наибольшей вероятностью принадлежит к «хорошим» результатам и с наименьшей — к «плохим».

0.7.3 Population Based Training (PBT)

Метод, вдохновленный эволюцией, идеально подходит для тяжелых нейросетей. В отличие от других методов, PBT подбирает параметры **прямо в процессе обучения**, а не запускает обучение заново.

- **Exploit:** Слабые модели в популяции «умирают», копируя веса более успешных соседей.
- **Explore:** После копирования весов гиперпараметры модели-победителя слегка мутируют (добавляется шум), чтобы найти еще более удачную комбинацию.
- **Результат:** Мы получаем и обученную модель, и оптимальное расписание (schedule) изменения параметров (например, learning rate) за один проход.

0.7.4 Hyperband и Successive Halving

Эти алгоритмы фокусируются на эффективном распределении ресурсов (времени, данных).

- **Принцип:** Мы запускаем много конфигураций на очень малом количестве итераций.
- **Отсев:** После каждой стадии выживает только топ-1/ n лучших конфигураций, которым выделяется в n раз больше ресурсов.
- Это позволяет быстро отбросить явно плохие архитектуры и сосредоточиться на перспективных.

0.8 Опенсорс-библиотеки и инструменты

Выбор инструмента зависит от сложности модели и имеющихся вычислительных ресурсов.

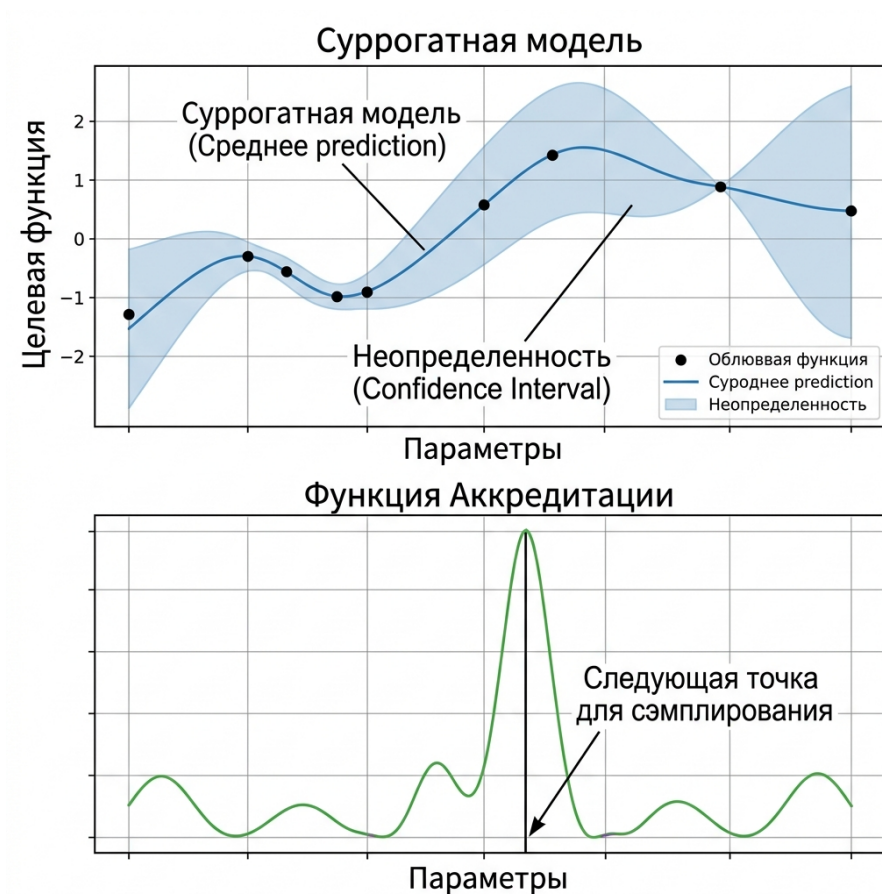


Рис. 3: Интуиция байесовской оптимизации.

Библиотека	Ключевые особенности и алгоритмы
Scikit-learn	Базовый стандарт. Предлагает <code>GridSearchCV</code> и <code>RandomizedSearchCV</code> . В новых версиях появился <code>HalvingGridSearchCV</code> , который использует метод последовательных сокращений (Successive Halving) для быстрой отбраковки плохих комбинаций.
Optuna	Лидер индустрии. Использует подход <i>Define-by-run</i> (пространство поиска описывается прямо внутри функции). • Pruning: Автоматически останавливает «безнадежные» итерации (trial), не дожидаясь конца обучения. • Алгоритмы: TPE, CMA-ES, Random Search. Отличная визуализация важности параметров.
Hyperopt	Классическая библиотека, популяризовавшая алгоритм TPE (Tree-structured Parzen Estimator). Имеет чуть более сложный синтаксис по сравнению с Optuna, но всё еще широко используется.

Scikit-Optimize	Специализируется на байесовской оптимизации . Главный плюс — наличие BayesSearchCV , который можно использовать как прямую замену стандартным методам из sklearn .
Ray Tune	Инструмент для масштабируемого подбора. Идеален для распределенных систем и тяжелых нейросетей. Поддерживает продвинутые стратегии, такие как PBT (Population Based Training) и Hyperband.
Keras Tuner	Специализированное решение для экосистемы TensorFlow/Keras. Позволяет подбирать не только параметры, но и саму архитектуру нейросети (количество слоев, фильтров и т.д.).

Antigravity: Начинай всегда с **Random Search**, чтобы быстро прощупать почву. Если задача критически важна — переходи на **Optuna**.